

딥러닝 프로그래밍

한국폴리텍대학 하이테크과정
2026년 2학기
박병주 강사



목차

A table of Contents

- 01** 지난 시간 복습
- 02** 퍼셉트론과 선형 계층 (Linear)
- 03** 활성화 함수 (ReLU, Sigmoid)
- 04** nn.Module로 나만의 MLP를 구현
- 05** 출력 계층과 shape 흐름을 설계
- 06** 직접 만든 MLP를 학습시키고 구조를 실험



PART 01

지난주 복습

1. 복습: 파이프라인은 완성



- 지난주까지 완성한 것
 - Dataset에 transform을 등록해 전처리를 자동화하고, DataLoader로 배치 공급을 표준화했다
 - train과 validation을 분리해 매 epoch 일반화 성능을 점검하며 94% 중반을 얻었다
- 오늘의 질문
 - 그런데 모델은 아직 `nn.Sequential(...)` 완성품을 상자째 쓰고 있다, 안에는 뭐가 있을까?

1. 오늘의 목표: 완성품을 부품으로 분해

1주차부터 상자째 쓴 모델, 오늘 분해한다

지금까지 완성품으로 쓴 모델 (1주차부터)

```
model = nn.Sequential(nn.Linear(784, 100), nn.ReLU(),  
                      nn.Linear(100, 10))
```

오늘 할 일:

1. `nn.Linear` 내부의 `weight`와 `bias`를 직접 확인한다

2. `nn.ReLU`가 왜 필요한지 실험으로 확인한다

3. 같은 모델을 `nn.Module` 클래스로 직접 구현

• 오늘이 중요한 이유

- CNN, 전이학습 모두 오늘 배우는 틀 위에 층만 바꿈
- PyTorch 실무와 논문 코드의 모델은 `nn.Module` 클래스로 작성하는 것이 표준

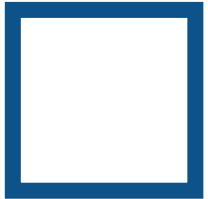
1. 환경 확인

확인 셀, 두 줄이 나오면 준비 끝

```
import torch
import torch.nn as nn          # 오늘의 핵심 모듈 (neural network)
device = "cuda" if torch.cuda.is_available() else "cpu"
print(torch.__version__)      # → (예) 2.13.0+cu126
print(device)                 # → (예) cuda   (GPU가 없으면 cpu)
```

- 지난주와 같은 환경

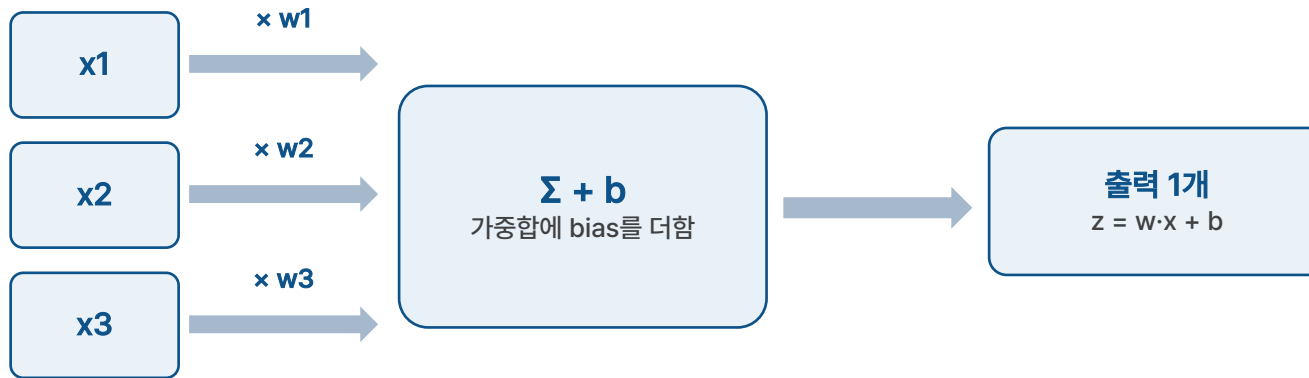
- `torch.nn`은 신경망 부품이 모여 있는 모듈이며,
`nn`으로 줄여 쓰는 것이 관례



PART 02

퍼셉트론과 선형 계층 (Linear)

2. 뉴런 하나의 계산



뉴런 하나 = 입력마다 weight를 곱해 더하고 bias를 더한다, 1주차 직선 $y = w \cdot x + b$ 그대로

- 읽는 법

- 뉴런 하나는 입력마다 weight를 곱해 더하고 bias를 더한 값 하나를 출력한다
- 1주차 직선 $y = w \cdot x + b$ 와 같은 계산이며, 입력이 여러 개로 늘었을 뿐이다
- 1958년 퍼셉트론은 이 가중합에 임계 판정을 붙인 것이었고, 현대 신경망은 판정 대신 활성화 함수를 쓴다

2. nn.Linear = 뉴런을 묶은 계층

구성	의미	Linear(784, 100)에서
weight	뉴런마다 입력 개수만큼의 가중치	shape (100, 784), (출력, 입력) 순서
bias	뉴런마다 1개의 기준값	shape (100,)
파라미터 수	weight 개수 + bias 개수	$784 \times 100 + 100 = 78,500$

- `nn.Linear(784, 100)`은 입력 784개를 받는 뉴런 100개를 한 계층으로 묶은 것
- 학습이 갱신하는 대상이 바로 이 weight와 bias다, 1주차의 w와 b가 78,500개로 늘어난 것

2. 예제 3-1: Linear 해부

예제 3-1. 선형 계층의 내부

```
import torch
import torch.nn as nn
torch.manual_seed(0)                # 초기값 재현용
layer = nn.Linear(784, 100)          # 입력 784 → 출력 100인 선형 계층
print(layer.weight.shape)            # → torch.Size([100, 784]) (출력, 입력) 순서
print(layer.bias.shape)              # → torch.Size([100])
n = sum(p.numel() for p in layer.parameters())
print(n)                             # → 78500 (784×100 + 100)
x = torch.randn(64, 784)            # 배치 64개
print(layer(x).shape)               # → torch.Size([64, 100])
```

• 확인 포인트

- weight는 (100, 784), (출력, 입력) 순서라는 관례에 주의
- 파라미터 수는 $784 \times 100 + 100 = 78,500$
- 배치 (64, 784)를 넣으면 (64, 100), 배치 차원은 유지되고 특징만 변환

2. weight 값 들여다보기

예제 3-1에 이어서, 실제 값 확인

```
print(layer.weight[0, :5])  # 0번 뉴런이 가진 가중치 앞 5개
# → tensor([-0.0003,  0.0192, -0.0294, -0.0263, -0.0138], grad_fn=...)
print(layer.bias[0])        # 0번 뉴런의 bias
# → tensor(-0.0354, grad_fn=...)
print(layer.weight.requires_grad)  # 학습으로 갱신되는 파라미터라는 표시
# → True
```

- 확인 포인트

- 학습 전이라 작은 무작위 값으로 초기화되어 있다
- `requires_grad`가 True다, 기울기를 계산해 학습으로 갱신할 수 있는 값이라는 표시다 (출력의 `grad_fn`은 값을 꺼낸 연산의 기록)
- 학습이 돌면 이 78,500개의 숫자가 조금씩 변한다, 그것이 모델이 배우는 실체다

2. 예제 문제, Linear 계층 진단

문제

1. `nn.Linear(20, 30)`의 파라미터 수는 몇 개일까?
2. 그 계층의 `weight.shape`은 (20, 30)일까, (30, 20)일까?
3. 입력으로 (64, 20) 배치를 넣으면 출력 shape은 무엇일까?

- 실행하지 말고 먼저 생각
- 784와 100을 20과 30으로 바꿨을 뿐입니다

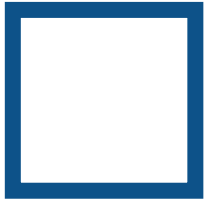
2. 예제 문제 풀이

실행해서 검증

```
import torch
import torch.nn as nn
layer = nn.Linear(20, 30)
print(sum(p.numel() for p in layer.parameters())) # 1.  $20 \times 30 + 30$ 
# → 630
print(layer.weight.shape) # 2. (출력, 입력) 순서
# → torch.Size([30, 20])
print(layer(torch.randn(64, 20)).shape) # 3. 배치는 유지, 특징  $20 \rightarrow 30$ 
# → torch.Size([64, 30])
```

• 풀이

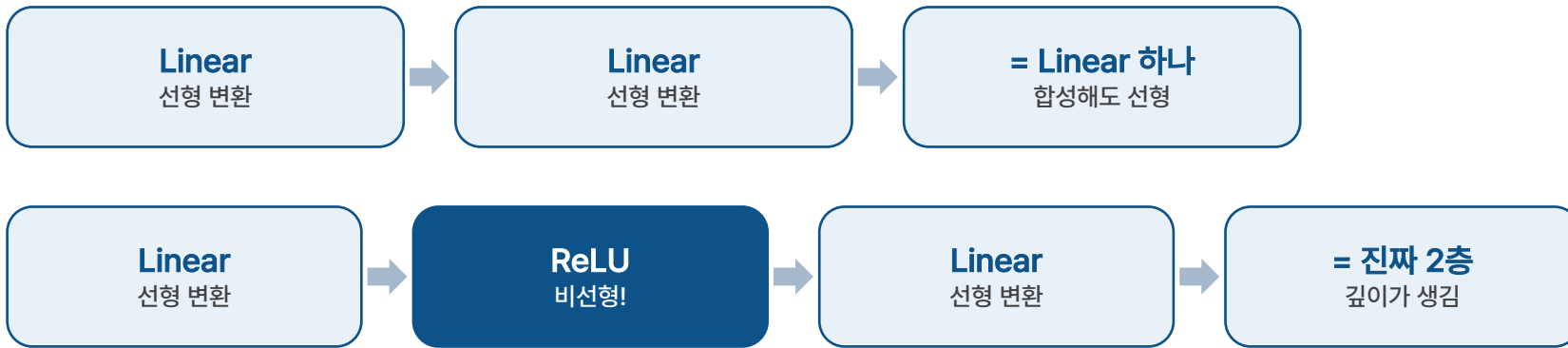
- 파라미터 수는 $20 \times 30 + 30 = 630$ 개
- weight는 (30, 20), PyTorch는 (출력, 입력) 순서로 저장한다
- 출력은 (64, 30), 배치 차원은 유지되고 특징 차원만 바뀐다



PART 03

활성화 함수 (ReLU, Sigmoid)

3. 왜 활성화 함수가 필요한가



- 선형만 쌓으면 생기는 일

- Linear를 연달아 쌓으면 수학적으로 **Linear** 하나와 같다, 직선을 두 번 그어도 직선인 것과 같다
- 계층 사이에 비선형 함수(활성화)를 끼워야 표현할 수 있는 함수가 늘어난다, deep의 조건
- 이후 실험으로 확인

3. ReLU와 Sigmoid

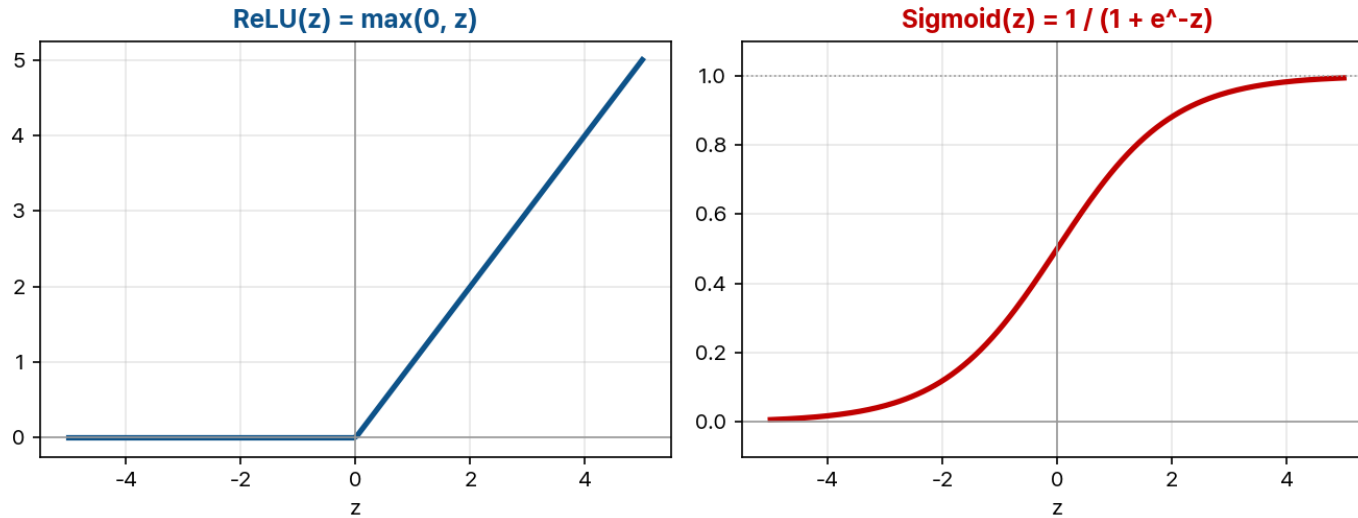


Fig. 1. 은닉 계층의 표준 ReLU, 확률 출력용 Sigmoid

- 두 함수의 용도

- ReLU는 음수를 0으로 바꾸는 단순한 함수지만, 계산이 싸고 학습이 잘 되어 은닉 계층의 표준이다
- Sigmoid는 값을 0~1로 압축한다, 초기 신경망의 표준이었으나 깊은 층에서 기울기 소실을 일으켜 은닉 계층에서는 밀려났다
- 그 문제가 초기 심층 신경망의 학습을 어렵게 했고, ReLU가 해결의 한 축이었다 (지금 Sigmoid는 이진 분류의 확률 출력 등에 쓴다)

3. 예제 3-2: 값으로 확인

예제 3-2. 활성화 함수에 값을 넣어 보면

```
import torch
z = torch.tensor([-2., -1., 0., 1., 2.])
print(torch.relu(z))          # 음수만 0으로, 양수는 그대로
# → tensor([0., 0., 0., 1., 2.])
print(torch.sigmoid(z))       # 어떤 값이든 0~1 사이로 압축
# → tensor([0.1192, 0.2689, 0.5000, 0.7311, 0.8808])
```

- 출력 해석
 - ReLU는 음수만 0으로 바꾸고 양수는 그대로 통과
 - Sigmoid는 어떤 값이든 0~1 사이로 눌러 담는다, 0은 정확히 0.5가 된다
- 기억할 것
 - 이 단순한 한 줄짜리 함수가 선형과 선형 사이에서 깊이를 만든다

3. 실습 1: 활성화 함수 그려 보기 (문제)

실습

1. `torch.linspace(-5, 5, 100)`으로 입력 구간 만들기
2. `torch.relu`와 `torch.sigmoid` 곡선을 `plt.plot`으로 겹쳐 그리기
3. `relu(-3.7)`과 `sigmoid(100.)`의 값 예측 후 확인

• 힌트

- `z = torch.linspace(-5, 5, 100)` 그리고 `plt.plot(z, torch.relu(z), label="ReLU")`
- 두 번째 곡선도 같은 방식으로 그린 뒤 `plt.legend()`와 `plt.show()`
- ③은 `torch.tensor(-3.7)`처럼 값 하나짜리 Tensor로 확인한다

• 확인

- 그래프가 앞 슬라이드의 곡선과 같은 모양이면 성공

3. 실습 1: 정답 예시

실습 1. 정답 예시

```
import torch
import matplotlib.pyplot as plt
z = torch.linspace(-5, 5, 100)      # 1. -5부터 5까지 100개 점
plt.plot(z, torch.relu(z), label="ReLU")      # 2. 두 곡선을 겹쳐 그리기
plt.plot(z, torch.sigmoid(z), label="Sigmoid")
plt.legend(); plt.grid(alpha=0.3); plt.show()
print(torch.relu(torch.tensor(-3.7)).item())  # 3. 음수는 전부 0
# → 0.0
print(torch.sigmoid(torch.tensor(100.)).item()) # 3. 1에 한없이 가까워 1.0으로 출력
# → 1.0
```

- 포인트

- `relu(-3.7)`은 **0.0**, 음수는 크기와 무관하게 전부 0이 된다
- `sigmoid(100.)`은 **1.0**으로 출력된다, 수학적으로는 1보다 아주 조금 작지만 표현할 수 있는 자릿수의 한계로 1이 된다
- 극단값을 넣어 함수의 성질을 확인하는 것은 좋은 검증 습관이다

- 다음

- 이 부품들을 클래스 하나로 조립한다



PART 04

nn.Module로 나만의 MLP를 구현

4. MLP와 두 가지 구현 방식

방식	형태	특징
<code>nn.Sequential</code>	부품을 일렬로 나열	간단하지만 흐름이 한 줄뿐
<code>nn.Module</code> 클래스	부품 선언과 흐름 정의를 분리	자유로운 구조 가능, PyTorch의 표준 방식

- 우리가 쓰던 모델의 정식 명칭이 **MLP**(Multi-Layer Perceptron), 선형 계층과 활성화를 겹겹이 쌓은 신경망이다
- `nn.Sequential`은 일렬 조립만 되지만, `nn.Module` 클래스는 데이터 흐름을 자유롭게 정의할 수 있다
- CNN부터 프로젝트까지 전부 이 방식으로 모델을 작성한다

4. 예제 3-3: MLP 클래스 작성

예제 3-3. 표준 틀, 빈칸은 부품과 흐름뿐

```
import torch
import torch.nn as nn

class MLP(nn.Module):
    # 이 클래스 구조는 표준 틀, 복사해서 시작
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(784, 100)
        self.act = nn.ReLU()
        self.fc2 = nn.Linear(100, 10)
        # 부품 선언: 은닉 계층
        # 부품 선언: 활성화
        # 부품 선언: 출력 계층
    def forward(self, x):
        # 데이터가 흐르는 순서를 정의
        return self.fc2(self.act(self.fc1(x)))

torch.manual_seed(0)
model = MLP()
print(sum(p.numel() for p in model.parameters())) # → 79510 (78500 + 1010)
print(model(torch.randn(64, 784)).shape) # → torch.Size([64, 10])
```

• 틀 읽는 법

- `__init__`에는 부품을 선언하고, `forward`에는 데이터가 흐르는 순서를 쓴다
- `class`와 `super().__init__()`은 틀의 고정 문구다, 복사해서 시작한다
- `model(x)`를 부르면 PyTorch가 알아서 `forward`를 실행한다

• 검수

- 파라미터 79,510개와 출력 (64, 10)이 완성품과 동일하다, 같은 것을 직접 만든 것

4. 실습 2: 은닉 계층 키우기 (문제)

실습

2. 예제 3-3을 복사해 은닉 뉴런을 100 → 256으로 바꾼 MLP256 클래스를 만든다.

1. fc1을 Linear(784, 256)으로 수정
2. fc2의 입력도 맞춰 바꾸기
3. 파라미터 수를 예측한 뒤 코드로 확인

• 힌트

- 변경 지점은 네 곳이다, fc1의 출력 숫자, fc2의 입력 숫자, 클래스 이름, 그리고 `model = MLP()` 줄의 클래스 이름
- 파라미터 예측은 $784 \times 256 + 256 + 256 \times 10 + 10$
- `model(torch.randn(8, 784)).shape`으로 출력이 (8, 10)인지 확인한다

• 확인

- fc2 입력을 안 바꾸면 실행할 때 **shape** 오류가 난다, 그 오류는 다음 파트에서 해부한다

4. 실습 2: 정답 예시

실습 2. 정답 예시

```
import torch
import torch.nn as nn

class MLP256(nn.Module):
    # 1. 은닉 뉴런을 100 → 256으로
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(784, 256)
        self.act = nn.ReLU()
        self.fc2 = nn.Linear(256, 10)    # 2. 출력 계층의 입력도 함께 256으로
    def forward(self, x):
        return self.fc2(self.act(self.fc1(x)))

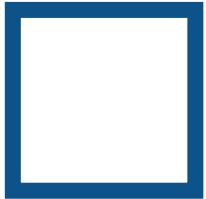
model = MLP256()
print(sum(p.numel() for p in model.parameters()))    # 3. 파라미터 수 확인
# → 203530    (784×256+256 + 256×10+10)
print(model(torch.randn(8, 784)).shape)    # → torch.Size([8, 10])
```

- 포인트

- fc1의 출력 256과 fc2의 입력 256은 연결된 통로라 반드시 함께 바꾼다
- 파라미터가 79,510 → 203,530으로 약 2.6배가 됐다

- 다음

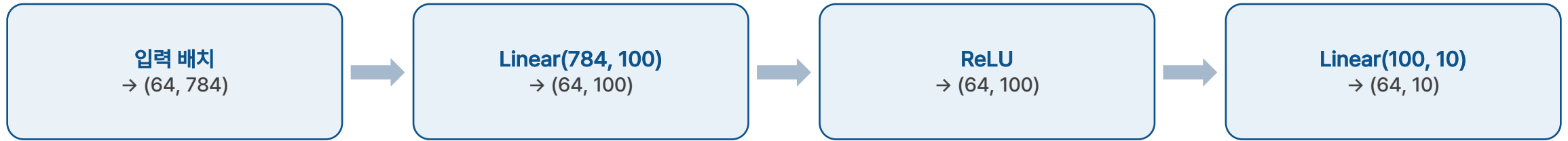
- 커진 모델이 성능도 좋을까?



PART 05

출력 계층과 shape 흐름을 설계

5. shape 흐름: 입구부터 출구까지



첫 차원(배치 64)은 끝까지 유지되고, 특징 차원만 784 → 100 → 10으로 변환된다

- 설계 규칙 세 가지

- 입구의 입력 수는 데이터의 특징 수와 같아야 한다 (MNIST는 $28 \times 28 = 784$)
- 출구의 출력 수는 클래스 수와 같아야 한다 (숫자 10종이라 10), 그 10개 값이 **logits**다
- 활성화는 shape을 바꾸지 않고, 배치 차원은 끝까지 유지된다

5. 예제 3-4: 흐름 추적과 오류 읽기

예제 3-4. 한 계층씩 통과시키며 shape 추적

```
import torch
import torch.nn as nn
torch.manual_seed(0)
fc1, act, fc2 = nn.Linear(784, 100), nn.ReLU(), nn.Linear(100, 10)
x = torch.randn(64, 784)          # 배치 64개가 들어와서
h = fc1(x); print(h.shape)         # → torch.Size([64, 100])   (특징 784 → 100)
h = act(h); print(h.shape)         # → torch.Size([64, 100])   (활성화는 shape을 안 바꿈)
out = fc2(h); print(out.shape)     # → torch.Size([64, 10])    (클래스 수 10)
print(out[0].argmax().item())      # 0번 샘플의 예측 = 10개 Logit 중 최댓값 위치
# → (예) 3   (아직 학습 전이라 무작위 수준)
```

잘못 쌓으면 나는 오류

```
# 순서를 잘못 쌓으면 (출력 계층을 먼저)
wrong = nn.Sequential(nn.Linear(100, 10), nn.ReLU(), nn.Linear(784, 100))
wrong(torch.randn(64, 784))
# → RuntimeError: mat1 and mat2 shapes cannot be multiplied (64x784 and 100x10)
# 1주차 오류 표의 그 오류, 입력 특징 수(784)와 계층의 입력(100)이 안 맞는다는 뜻
```

- 왜 이렇게 확인하나

- 모델이 복잡해져 막히면 한 계층씩 통과시키며 shape을 찍는 것이 실전 디버깅이다
- 1주차의 '막히면 shape부터'가 모델 안으로 들어온 것이다

- 아래는 일부러 낸 오류

- 메시지의 두 shape을 읽으면 입력 784와 계층 입력 100이 안 맞는다는 것이 바로 보인다

5. 예제 문제, shape 설계

문제

1. `nn.Linear(784, 100)`에 배치 (32, 784)를 넣으면 출력 shape은?
2. CIFAR-10의 컬러 이미지 (3, 32, 32)를 펴서 넣으려면 첫 계층의 입력 수는 몇이어야 할까?
3. 알파벳 26종을 분류하는 모델이라면 마지막 계층의 출력 수는?

- 오늘 배운 설계 규칙 세 가지(입구, 출구, 배치 유지)로 전부 풀립니다

5. 예제 문제 풀이

실행해서 검증

```
import torch
import torch.nn as nn
layer = nn.Linear(784, 100)
print(layer(torch.randn(32, 784)).shape)  # 1. 배치 32도 문제없음
# → torch.Size([32, 100])
# 2. CIFAR-10 컬러 이미지 (3, 32, 32)라면 특징 수 = 3×32×32
print(3 * 32 * 32)
# → 3072 (첫 계층은 nn.Linear(3072, ...)로)
```

• 풀이

- 출력은 (32, 100), 배치가 몇이든 규칙은 같다
- 컬러는 채널 3장이므로 $3 \times 32 \times 32 = 3072$, 첫 계층은 `nn.Linear(3072, ...)`
- 클래스가 26종이면 마지막 계층의 출력은 26, 출구 규칙 그대로다



PART 06

직접 만든 MLP를 학습시키고 구조를 실험

6. 예제 3-5: 파이프라인에 연결

예제 3-5. 바뀐 것은 `model = MLP()` 한 줄뿐

```
# 2주차 파이프라인은 그대로 (train_loader, val_loader)
torch.manual_seed(0)
model = MLP().to(device)          # 오늘 직접 구현한 그 클래스
opt = torch.optim.SGD(model.parameters(), lr=0.1)
loss_fn = nn.CrossEntropyLoss()
for epoch in range(3):
    for x, y in train_loader:      # 2주차 학습 루프 그대로
        x, y = x.view(-1, 784).to(device), y.to(device)
        loss = loss_fn(model(x), y)
        opt.zero_grad(); loss.backward(); opt.step()
    correct = 0
    with torch.no_grad():
        for x, y in val_loader:
            x, y = x.view(-1, 784).to(device), y.to(device)
            correct += (model(x).argmax(dim=1) == y).sum().item()
    print(epoch + 1, round(correct / 10000 * 100, 1))
# → (예) 1 89.9
# → (예) 2 93.8
# → (예) 3 94.9   (같은 환경이면 2주차 완성품과 똑같다)
```

• 포인트

- 2주차 파이프라인과 학습 루프는 한 줄도 바뀌지 않았다
- 이제 구조를 바꿀 자유가 생겼다, 바로 실험한다

6. 구조 실험: ReLU를 빼면

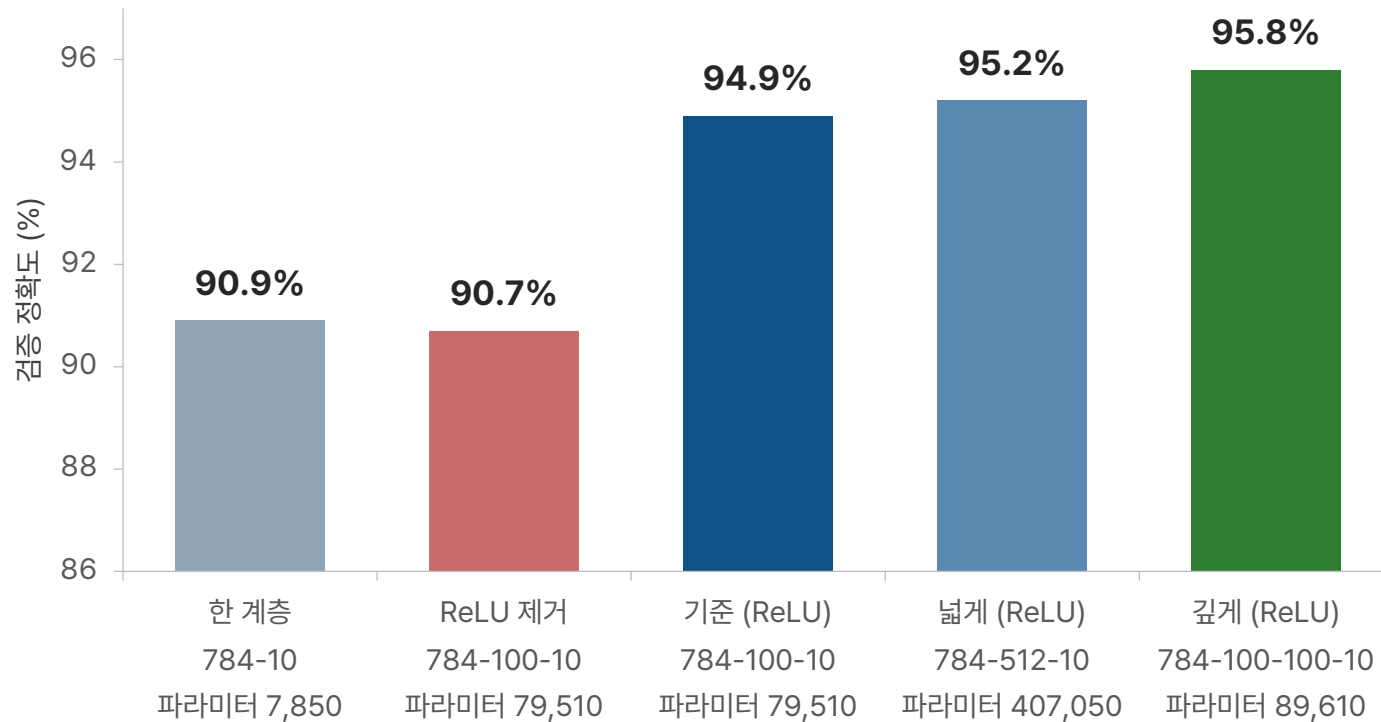


Fig. 2. 같은 조건에서 구조만 변경한 3 epoch 검증 정확도 (단일 실행 실측, 넓게는 512 기준)

• 그래프 해석

- 이 실험에서 ReLU를 뺀 모델(파라미터 79,510)은 한 계층 모델(7,850)과 사실상 같은 성능
- 선형만 쌓으면 표현력이 한 계층과 같다는 설명이 실험 결과로도 확인 가능
- 이 실험에서는 깊게 쌓기(95.8%)가 넓게 만들기(95.2%)보다 파라미터 효율이 좋았다

6. 실습 3: 구조 실험 (문제)

실습

예제 3-3의 클래스와 3-5의 학습을 복사해 구조를 바꿔 기록한다.

1. forward에서 self.act를 빼 ReLU 제거
2. fc 계층을 하나 더 넣어 784-100-100-10으로
3. 실습 2의 MLP256으로 학습

- 클래스 이름을 바꿨다면 예제 3-5의 `model = MLP().to(device)`도 그 이름으로 바꿔야 실험이 된다

6. 실습 3: 정답 예시

실습 3. 관찰 정리

- # 1. ReLU 제거: (예) 약 90.7%, 한 계층(90.9%)과 사실상 동일
(선형 계층만 쌓으면 수학적으로 한 계층과 같다, 비선형이 깊이를 만든다)
- # 2. 깊게 784-100-100-10: (예) 약 95.8%, 파라미터 +10,100개로 +0.9%p
- # 3. 넓게 784-256-10 (MLP256): (예) 2 epoch 93.9% → 3 epoch 93.0%
(epoch마다 오르내림이 커서 마지막 값 하나로는 판단하기 어렵다)

• 관찰 결과

- ReLU를 빼면 한 계층 모델과 같은 수준으로 떨어진다, 비선형이 깊이를 만든다
- 이 실험에서는 깊게 쌓은 모델이 더 적은 파라미터로 넓은 모델(512)보다 높았다
- MLP256은 2 epoch 93.9%에서 3 epoch 93.0%로 떨어졌다, 마지막 epoch 하나만 보면 흔들린다
- 구조를 체계적으로 고르는 방법은 구조 튜닝에서 다룬다

6. 예제 문제, 구조 설계 진단

문제

1. 클래스가 100종인 분류 모델이라면 마지막 계층은 어떻게 선언해야 할까?
2. nn.Linear 5개를 활성화 없이 쌓으면 한 계층 모델보다 더 복잡한 패턴을 표현할 수 있을까?
3. 파라미터가 많은 모델이 항상 더 좋은 성능을 낼까?

- 오늘 수업 전체가 이 세 문제에 들어 있습니다

6. 예제 문제 풀이

문제	풀이	답
1	출구 규칙, 출력 뉴런 수는 클래스 수와 같다	<code>nn.Linear(앞 계층 출력 수, 100)</code>
2	활성화가 없으면 Linear 5개도 하나의 Linear로 합쳐진다	아니다, 표현력은 한 계층과 같다
3	ReLU 없는 79,510개 모델이 7,850개 모델과 동급이었다	보장하지 않는다

- 오늘 실험이 근거다, 수를 늘리는 것보다 구조를 올바르게 만드는 것이 먼저다

6. 정리, 과제, 다음 주

오늘의 구성 요소	역할	핵심 한 줄
<code>nn.Linear(in, out)</code>	선형 계층 (뉴런 묶음)	<code>weight (out, in) + bias</code> , 파라미터의 실체
<code>nn.ReLU()</code>	활성화 함수	음수만 0으로, 깊이를 만드는 조건
<code>nn.Module</code> 클래스	모델 구현의 표준 틀	<code>__init__</code> 부품 선언, <code>forward</code> 데이터 흐름
설계 규칙	입구와 출구	입력 수 = 특징 수, 출력 수 = 클래스 수

- 다음 주, 학습 과정의 정식 정리

- `loss`와 `backpropagation`, `optimizer`가 정확히 무엇을 하는지 열어 보고, 학습 곡선으로 상태를 진단한다